CONTROL DE CAMBIOS - REFACTORIZACIÓN PRINCIPIOS SOLID

CC-01: Violación del Principio Abierto/Cerrado (OCP)

Campo	Valor
ID	CC-01
Archivo / Clase Modificada	`src/modules/work-orders/work-orders.service.ts` - Clase `WorkOrderService`
Fallo SOLID / Antipatrón	Violación del Principio Abierto/Cerrado (OCP - Open/Closed Principle)
Descripción del Problema	El servicio `list()` verifica manualmente el rol del usuario con comparaciones de texto hardcodeadas (`if (userRole === 'tecnico')`). Si se agregan nuevos roles (supervisor, auditor, etc.), el código central debe modificarse, violando OCP. La lógica de autorización está mezclada con la lógica de negocio, lo que reduce la encapsulación y dificulta el escalado de roles.
Mejora Aplicada	Se implementó el patrón **Strategy** mediante una interfaz `IAuthorizationFilter` que encapsula la lógica de filtrado específica por rol. Cada rol tiene su propia estrategia de filtrado. El servicio delega al objeto estrategia sin conocer detalles de implementación. Nuevos roles pueden agregarse sin modificar el servicio: solo se crea una nueva estrategia que implementa la interfaz.
Severidad	**A (Alta)** - Impide escalar roles de forma cerrada; requiere modificación de código central cada vez.

Evidencia ANTES (Violación de OCP):

```
async list(options: {
  status?: OrderStatus;
  userRole?: string;
  userTechnicianId?: string;
}): Promise<{ items: WorkOrder[]; total: number }> {
  // Regla de negocio y roles quemados dentro del servicio
  if (options.userRole === 'tecnico' && !options.technicianId) {
    options.technicianId = options.userTechnicianId;  // ❌ Hardcodeado
  }
  // Si mañana hay "supervisor", "auditor", etc., hay que modificar aquí

  const { rows, total } = await this.repository.list(options);
  return { items: rows.map(WorkOrder.fromRow), total };
}
```

Problema:

- El código está **abierto a modificación** (requiere editar el método `list()`)
- Cada nuevo rol requiere agregar un `if` más
- Violación de OCP: "Software entities should be open for extension, closed for modification"

Evidencia DESPUÉS (Implementación de Strategy Pattern + OCP):

1. Interfaz Strategy para Autorización:

```
// src/shared/authorization/filters/authorization-filter.interface.ts
export interface IAuthorizationFilter {
  applyFilter(options: ListOptions): void;
}
```

2. Estrategia para cada rol:

```
// src/shared/authorization/filters/technician-authorization-filter.ts
export class TechnicianAuthorizationFilter implements IAuthorizationFilter {
  applyFilter(options: ListOptions): void {
    if (!options.technicianId && options.userTechnicianId) {
      options.technicianId = options.userTechnicianId;
    }
  }
}

// src/shared/authorization/filters/admin-authorization-filter.ts
export class AdminAuthorizationFilter implements IAuthorizationFilter {
  applyFilter(options: ListOptions): void {
    // Admin ve todas las órdenes, no aplica filtro
  }
}
```

3. Factory para crear estrategias:

```
// src/shared/authorization/filters/authorization-filter-factory.ts
export class AuthorizationFilterFactory {
  static createFilter(role: string): IAuthorizationFilter {
    const filters: Record<string, IAuthorizationFilter> = {
      'tecnico': new TechnicianAuthorizationFilter(),
      'administrador': new AdminAuthorizationFilter(),
      'cliente': new ClientAuthorizationFilter(),
      // Fácil de extender: agregar 'supervisor', 'auditor', etc.
    };

    return filters[role] || new ClientAuthorizationFilter(); // Default
  }
}
```

4. Servicio refactorizado (OCP compliant):

```
// src/modules/work-orders/work-orders.service.ts
async list(options: {
  status?: OrderStatus;
  userRole?: string;
  userTechnicianId?: string;
}): Promise<{ items: WorkOrder[]; total: number }> {
  // Obtener estrategia según el rol (sin conocer detalles)
```

```
  const filter = AuthorizationFilterFactory.createFilter(options.userRole || 'cliente');

  // Aplicar filtro sin saber qué rol es
  filter.applyFilter(options);

  // Resto del código igual
  const { rows, total } = await this.repository.list(options);
  return { items: rows.map(WorkOrder.fromRow), total };
}
```

Beneficios de la Mejora:

Aspecto	Antes	Después
Escalabilidad	❌ Modificar servicio	✅ Crear nueva estrategia
OCP	❌ Abierto a modificación	✅ Cerrado a modificación
Testabilidad	❌ Difícil mockear roles	✅ Fácil inyectar estrategia
Responsabilidad	❌ Servicio + autorización	✅ Servicio solo lógica negocio
Mantenibilidad	❌ Cambios en core	✅ Cambios en interfaces

Patrón Aplicado:

- **Strategy Pattern**: Cada rol es una estrategia intercambiable
- **Factory Pattern**: Creación centralizada de estrategias
- **Dependency Inversion**: El servicio depende de la interfaz, no de implementaciones concretas

---

Cómo Agregar un Nuevo Rol (sin modificar el servicio):

```
// 1. Crear nueva estrategia
export class SupervisorAuthorizationFilter implements IAuthorizationFilter {
  applyFilter(options: ListOptions): void {
    // Lógica específica de supervisor
  }
}

// 2. Registrar en factory (única línea)
filters['supervisor'] = new SupervisorAuthorizationFilter();

// 3. ¡Listo! El servicio NO se modifica
```

✅ **Código cerrado a modificación, abierto a extensión**

Resumen de Cambios

Antes	Después
Verificación de rol hardcodeada en servicio	Estrategia por rol en interfaces
Requiere modificar `WorkOrderService` para agregar rol	Solo crea nueva clase Strategy
Lógica de autorización mezclada con negocio	Autorización separada y reutilizable
Difícil de testear y mockear	Fácil inyectar estrategias mock

Archivos Modificados y Creados

✅ CREADOS (Nuevas estrategias + Factory):

1. `src/shared/authorization/filters/technician-authorization-filter.ts` ✅
   - Implementa filtrado para técnicos (auto-asignan su ID)
2. `src/shared/authorization/filters/admin-authorization-filter.ts` ✅
   - Implementa filtrado para administradores (sin restricciones)
3. `src/shared/authorization/filters/client-authorization-filter.ts` ✅
   - Implementa filtrado para clientes (solo sus órdenes)
4. `src/shared/authorization/filters/authorization-filter-factory.ts` ✅
   - Factory pattern que crea estrategias según rol
   - Centraliza mapeo de roles → estrategias
   - Facilita agregar nuevos roles sin modificar servicio

✅ MODIFICADOS (Refactorización con OCP):

1. `src/modules/work-orders/work-orders.service.ts` ✅
   - **ANTES**: `if (userRole === 'tecnico' && !options.technicianId) { ... }`
   - **DESPUÉS**: `const authFilter = AuthorizationFilterFactory.createFilter(options.userRole); authFilter.applyFilter(options);`
   - Ahora usa Strategy pattern delegando al factory
2. `src/modules/work-orders/work-orders.controller.ts` ✅
   - Agrega `userCustomerId` al llamar al servicio.list()
   - Soporta filtrado por cliente (ClientAuthorizationFilter)
3. `src/modules/work-orders/work-orders.repository.ts` ✅
   - Actualiza interfaz para aceptar `customerId` en options.list()
   - Implementa filtrado por `customer_id` en query

Estado: ✅ **COMPLETADO Build Status**: ✅ TypeScript + npm run build → SUCCESS **Patrón Implementado**: Strategy Pattern + Factory Pattern **Resultado**: OCP = ✅ CUMPLIDO (código cerrado a modificación, abierto a extensión)